DOKUMENTÁCIA ZÁPOČTOVÉHO PROGRAMU PAC-MAN

Obsah

1. Stručné zadanie
2. Presné zadanie
3. Čo program robí – ako ho používať?
4. Mechanika duchov
5. Kód

1. Stručné zadanie

Program je implementácia klasickej hry Pac-Man.

2. Presné zadanie

Program je implementácia klasickej hry Pac-Man pomocou C# a Windows Forms, hra je ovládaná šípkami na klávesnici.

3. Čo program robí – ako ho používať?

Hru spustíme tlačidlom Start Game, a ovládame ju šípkami na klávesnnici. Pac-Man zmení smer, aj keď šípku stlačíme skôr, ako v momente, kedy sad á otočiť, klávesu si totiž zapamätá. Cieľom je zjesť všetky bodky v bludisku, a získať za nich čo najväčšie skóre. Akonáhle sa tak stane, obrazovka sa resetuje, bodky sa doplnia a je to to isté odznova. Celý čas nás pritom naháňajú štyria duchovia, ktorý sledujú náš pohyb a snažia sa k nám dostať. V momente keď sa duch nachádza na rovnakom políčku ako my, prídeme o život (životy sú zobrazené vpravo hore) a po resetovaní pozícií pokračujeme. Hra končí, keď prídeme o všetky tri životy.
Na mape sa takisto nachádzajú väčšie bodky, ktoré keď zjeme, budeme mať na zopár sekúnd zrýchlenú rýchlosť pohybu, a takisto možnosť jesť duchov (ktorý budu od nás utekať) za skóre. Po zjedení sa duch vráti do stredu a správa sa ako normálny duch.
Skórovanie – za malú bodku 10 bodov, za veľkú bodku 50 bodov, za ducha 200, a za každého ďalšieho sa počet zdvojnásobuje (400, 800, 1600).
Na bokoch bludiska sa nachádza otvor, cez ktorý sa dá premiestniť na druhú stranu.

4. Mechanika duchov

Mechanika duchov je rozdielna od originálnej hry. Každý duch používa rovnaký algoritmus, ako sa pohybuje. Pri každom pohybe, keď sa duch dostane na križovatku, alebo odbočku, sa náhodne rozhodne, či si vyberie náhodný smer, alebo bude nasledovať hráča. Ak sa rozhodne nasledovať hráča, pustí algoritmus BFS po bludisku, a smer, v ktorom by sa k nemu dostal najrýchlejšie, je smer ktorý si zvolí. Táto mechanika vytvára pocit “náhodnosti”, no stále sa duchovia budú k hráčovi blížiť. Po zjedení veľkej bodky, keď duchovia pred hráčom utekajú, je pustený ten istý algoritmus, až na to že šanca, že si vyberú náhodný smer je menšia, a po nájdení ideálneho smeru k hráčovi si duch vyberie práve ten opačný smer.

V reálnej hre sa tento algoritmus nepoužíva, tam má každý duch vlastnú “osobnosť” a vlastné pravidlá pohybu, no ja som chcel skúsiť niečo vlastné, preto som si vybral tento algoritmus.

5. Kód

Po spustení programu sa vytvorí objekt Form1, ktorý bude predstavovať okno hry. Tento kód je vytvorený pomocou Windows Forms, a bude slúžiť iba na vytvorenie komponentov štartovacej obrazovky, ako Button a Label. Tiež zmeníme ikonu a názov okna. Buttonu pridáme funkciu button1_click, ktorá hru spustí. Po stlačení sa funkcia (v súbore Form1.cs) spustí, čím sa vytvorí objekt Graphics na kreslenie, spustí sa časovač, zmaže sa Button a Label, a vytvorí sa objekt Map (zo súboru Game.cs). Mapa si pamätá stav hry (State), podľa ktorej sa bude vykonávať hlavný loop hry, vo funkcii timer1_tick.

V classe Map si udržiavame:

- plán mapy, jeho šírku a výšku
- momentálny stav hry
- momentálne stlačenú šípku
- textúry
- časovač
- objekt hráča
- zoznam objektov duchov
- skóre, počet životov, zostávajúci počet bodiek na zjedenie

V classe Map budú nasledovné funkcie:

- konštruktor, spustí funkcie loadIcons, loadMap, nastaví stav na ready, a pre všetkých duchov pustí funkciu decideDirection
- loadMap, prečíta súbor map.txt, kde je uložený plán mapy, a načíta ho do plánu v objekte Map. Popritom vytvorí objekt hráča, objekty duchov, a spočíta všetky bodky, ktoré treba zjesť
- loadIcons, načíta štvorcové textúry, uložené v obrázku icons.png veďla seba, a uloží ich do bitmapy
- isWall, na rozhodnutie či je políčko stena
- changePosition, zmení pozíciu hráča na plane, prípadne ho teleportuje, takisto starú pozíciu označí ako prázdnu a skontroluje, či hráč nezjedol bodku, a či prípadne ich nezjedol už všetky
- startReverseMode, po zjedení veľkej bodky zmení stav hry, nastaví časovač, a zmení rýchlosti hráčovi a duchom
- timerChange, zmení časovač, používaný na meranie času v hre, a jeden tik. Skontroluje či vypršal, a ak áno, tak buď zmení stav späť do normálu, alebo v prípade začiatku hry hru začne
- Draw, vykreslí jeden frame hry. Najprv vykreslí pozadie (stney, bodky), potom hráča, potom duchov, potom skóre a životy, a potom prípade text ready alebo game over, podľa stavu hry. Pozícia vykresleného obrázku je daná buď fixne, alebo podľa pozície objektu na plane mapy. Pre hráča ešte musíme vypočítať, do akého smeru ide, a zakomponovať striedanie otvorenia úst pomocou offset pozície. Pri duchoch takisto treba

zakomponovať jeho smer, a pri obrátenom móde ešte zakomponovať blikanie jeho textúry.

- MoveEverything, funkcia volaná v hlavnom loope, načíta stlačenú klávesu, pohne hráčom a duchmi, a skontroluje kolíziu hráča s duchmi
- checkCollision, v prípade kolízie s normálnym duchom buď resetuje hru (vytvorí nových duchov a hráča), alebo vyhlási Game Over a hra konči, a v prípade zjedenia ducha, pripočíta skóre a ducha resetuje a vráti späť do stredu.

Objekt MovingObject je rodičovská abstraktná trieda, obsahuje referenciu na mapu, koordináty, smer pohybu, originálne kooridnáty, offsety na vykresľovanie, rýchlosť, funckiu Move a funkciu CanMove.

Objekt Hero obsahuje aj desiredX, desiredY, kde si pamätá, čo naposledy užívateľ stlačil, a teda kam sa má najbližšie otočiť (to sa deje vo funkcii keys). Vo funkcii Move sa najprv pokúsime zmeniť smer podľa desiredX, desiredY, následne sa pozrieme, či sa dá pohnúť smerom ktorým ideme. Ak áno, tak zmeníme offsety, a ak offset prekračuje hranicu 1, tak hráča pohneme do novej pozície (zavoláme map.changePosition).

Objekt Ghost obsahuje bool normal, ktorý hovorí o tom, v akom stave je duch. Funkcia Move funguje podobne ako pri hráčovi, bez desiredX, desiredY. Namiesto toho má duch funkciu decideDirection, pomocou ktorej sa na každom políčku rozhodne, či zmení svoj smer, a ak áno, tak kde.

Funkcia decideDirection sa najprv podľa náhodného čísla rozhodne, či si vyberie náhodný smer, alebo presný smer. Ak náhodný, tak si vyberie jeden zo štyroch vektorov, a ak sa vie v tomto smere pohnúť, a nie je to opačný smer akým ide teraz, tak si tento smer vyberie. V prípade presného smeru, si spustíme algoritmus BFS.

BFS algoritmus, prechádza plánom mapy, a hľadá pozíciu hráča. Pri každej pozícii si pamätáme počiatočný smer, ktorým sa tam dostaneme. Implementácia je pomocou fronty. Najprv do nej pridáme susedov ducha, a následne pustíme loop. Najprv skontrolujeme, či sme nenašli hrdinu, v tom prípade meníme smer ducha, ak nie, tak prejdeme všetkými jeho susedmi, a ak nie sú stenou, alebo sme túto pozíciu už neprešli (skontrolujeme funckiou haveIFound), tak pozíciu pridáme do fronty. Zakomponujeme tu aj teleportáciu po bokoch.